

HAUSARBEIT
Leonhard Lüdeke

Ein KI-Gamebot für Vier Gewinnt. Implementiert in Erlang

FAKULTÄT INFORMATIK UND DIGITALE GESELLSCHAFT

Faculty of Computer Science and Digital Society

Betreuung durch: Prof. Dr. Christoph Klause
Eingereicht am: 23. Januar 2026

HOCHSCHULE FÜR ANGEWANDTE
WISSENSCHAFTEN HAMBURG
Hamburg University of Applied Sciences

Inhaltsverzeichnis

1	Einleitung	2
1.1	Das Spiel Vier Gewinnt	2
2	Lösungsansätze	3
2.1	Minimax-Algorithmus	3
2.2	Alpha-Beta-Pruning Optimierung	4
2.3	Monte-Carlo-Treesearch	4
2.4	Wahl des Algorithmus	5
3	Entwurf	6
4	Realisierung des Entwurfs	6
5	Analyse	6
6	Fazit	6
	Literatur	6
	Selbstständigkeitserklärung	8

Diese Hausarbeit, die im Rahmen des Moduls Algorithmen und Datenstrukturen entstanden ist, beschäftigt sich mit dem Entwurf eines Gamebots für das Spiel Vier Gewinnt. Zu Beginn werden das Spiel und seine Regeln erklärt sowie bekannte Ansätze beleuchtet, die sich als Gamebot für Vier Gewinnt eignen. Anschließend wird ein Entwurfsmuster entwickelt, welches den ausgewählten Algorithmus (Gamebot) mit den gegebenen Codevorgaben (Das Spielbrett ist vorgegeben) darstellt. Darauf folgt die Beschreibung der Implementierung des Entwurfes, und eine Analyse des erstellten Codes.

Keywords: Vier Gewinnt, KI-Gamebot, Erlang

1 Einleitung

1.1 Das Spiel Vier Gewinnt

Vier Gewinnt ist ein klassisches Strategiespiel, und wurde 1973 von Howard Wexler handelt sich um ein endliches Zwei-Personen Nullsummenspiel mit perfekter Information. Ein Nullsummenspiel kennzeichnet sich dadurch aus, dass die Verluste des/der einen Spielers/in exakt den Gewinnen des/der anderen darstellen. Eine spielende Person ist deshalb perfekt Informiert, da die gesamte Spielsituation jederzeit offen einsehbar ist. Somit kann ein/e Spieler/in eine voll informierte Entscheidung treffen. [5]

Das Spiel wird auf einem vertikal aufgerichteten Brett mit sieben Spalten und sechs Zeilen gespielt (7x6). Die Spielenden werfen abwechselnd Steine ihrer Farbe von oben in eine Spalte ein, wobei der Stein auf die unterste freie Position fällt. Dabei stehen jeder Person 21 Spielsteine zur Verfügung. Gewinner ist diejenige Person, die zuerst vier eigene Steine horizontal, vertikal oder diagonal in einer Linie anordnet. [7]

Die Kombination aus einfacher Spielmechanik und überraschend hohen Gesamtzahl der legalen Spielpositionen von circa 4,5 Billionen [2], während Tic-Tac-Toe 5.478 gültige Positionen besitzt. Daher ist ein Aufbau eines Kompletten Spielbaumes sehr aufwendig, mit 4×10^{21} Blattknoten. Zusätzlich ist die Laufzeit, für die Berechnung des optimalen Spielzugs, in diesem Spielbaum praktisch unbrauchbar, wenn auch potenziell perfekt. Das ermitteln des Perfekten Zuges, für einen Gamebot, liegt somit nicht in einer 1 zu 1 Abbildung aller Spielzustände, sondern in Ansätzen, welche ich im nächsten Unterkapitel beleuchtet werden.

An dieser Stelle sei erwähnt das Victor Allis [1] und zeitgleich auch James D. Allen [1] 1988 das spiel vollständig lösten. Beide sind zu dem Ergebnis gekommen das die beginnende Person, bei perfektem Spielverhalten, immer gewinnt wenn sie den ersten Stein in die mittlere Spalte einwirft. Wenn nicht kommt endet das Spiel mit einem Unentschieden.

2 Lösungsansätze

Wie oben beschrieben ist es, aufgrund des großen Suchraumes, unpraktikabel alle Spielstände abzubilden. Der Unterschied, der einzelnen Lösungsansätze, liegt alleine darin wie dieser Zustandsraum durchsucht wird und wie der Suchbaum aufgebaut wird. Alle Verfahren haben dabei gemein, dass sie den Zustandsraum mit Knoten (Stellungen im Spiel) und Kanten (ein Zug) darstellen. Nachfolgend sind Knoten und Kanten als diese Zustände zu verstehen. Je nachdem welcher Algorithmus beschrieben wird, können diese Zustände zusätzliche Eigenschaften bekommen.

2.1 Minimax-Algorithmus

Ein fundamentales Konzept zur Berechnung von optimalen Strategien in Nullsummenspielen ist der Minimax-Algorithmus [6]. Dieser Algorithmus basiert auf einem Spielbaum, in dem jeder Knoten ein MAX- oder MIN-Knoten ist. Diese geben, neben dem Spielzustand, an welcher Person (Maxi- oder Minimierer/in) am Zug ist und besitzen einen numerischen Wert. Dieser numerische Wert gibt an welcher Spielzustand besser für die jeweilige Person ist. Der sogenannte Maximierer/in (meist 1. Spieler/in) wählt immer den Spielzustand mit dem höheren Wert und der Minimierer/in (meist 2. Spieler/in), den kleineren Wert. Dies entspricht einer Optimalen Spielweise. Werte gegen Null deuten auf eine ausgeglichene Spielsituation hin, negative Werte gereichen zum Vorteil für Minimierer/in und positive Werte für Maximierer/in.

Die Werte haben ihren Ursprung in den Blattknoten (Endzustände) des Baumes. Diese besitzen eine Bewertungsfunktion, die aussagt wie gut oder schlecht dieser Endzustand für die Maximierende Person ist. Somit setzen sich die Werte für die Inneren Knoten aus Ihren Kindknoten zusammen, also durch das Propagieren der Werte Ihrer Kindknoten. Dabei gilt für die Maximierenden Knoten $Wert = Maximum\ aller\ Kindwerte$ und für die Minimierenden Knoten $Wert = Minimum\ aller\ Kindwerte$. Die Vorgehensweise mittels negativ Bewertungen wird die Negamax-Variante genannt.

Klassisch wird dieser Algorithmus mittels einer Tiefensuche implementiert und besitzt ohne Optimierung eine exponentielle Laufzeit von $O(b^d)$ wobei $b = \text{mögliche Spalten pro Zug}$ und $d = \text{Züge im Voraus}$ darstellen. Aufgrund dieser großen Laufzeit ist eine Verbesserung notwendig, solche eine Optimierung bietet die Alpha-Beta Pruning Technik, welche nachfolgend beschrieben wird.

2.2 Alpha-Beta-Pruning Optimierung

Die Alpha-Beta-Pruning-Technik ist eine Optimierungsmethode des Minimax-Algorithmus, die durch intelligentes Abschneiden von Spielbaumästen die Anzahl der zu evaluierenden Knoten erheblich reduziert [3]. Das Verfahren arbeitet mit zwei Wertschranken:

- **Alpha** (α): Die beste (höchste) Bewertung, die der Maximierer bisher gefunden hat.
- **Beta** (β): Die beste (niedrigste) Bewertung, die der Minimierer bisher gefunden hat.

Ein Ast kann abgeschnitten werden, wenn $\alpha \geq \beta$, da die restlichen Züge in diesem Ast nicht mehr relevant für die optimale Spielentscheidung sind. Dies basiert auf der Annahme, dass beide Spieler optimal spielen und keine schlechteren Positionen akzeptieren würden [3].

Die Effektivität von Alpha-Beta-Pruning hängt stark von der Reihenfolge ab, in der die Züge evaluiert werden. Mit optimaler Zugreihenfolge ist eine Reduktion der Komplexität auf $O(b^{d/2})$ möglich, was eine quadratische Verbesserung gegenüber dem klassischen Minimax darstellt [3].

2.3 Monte-Carlo-Treesearch

Ein alternatives Konzept zur Berechnung von annähernd optimalen Strategien in Nullsummenspielen ist die Monte-Carlo-Baumsuche (MCTS). Wie der Minimax-Algorithmus basiert auch MCTS auf einem Spielbaum, in dem jeder Knoten einen Spielzustand repräsentiert und einen numerischen Wert besitzt. Im Gegensatz zum Minimax-Algorithmus, der durch statische Bewertungsfunktionen an den Blattknoten operiert, bestimmt MCTS diese Werte durch wiederholte stochastische Simulationen (Rollouts) bis zu den Endzuständen des Spielbaums. Durch die Propagierung der Simulationsergebnisse von den Blattknoten zu inneren Knoten konvergiert die Schätzung der Knotenwerte mit hinreichend vielen Iterationen gegen gute Näherungen der optimalen Spielqualität [4].

Die MCTS-Methode durchläuft zur Wertberechnung vier konzeptionelle Phasen: In der Selection-Phase werden bestehende Knoten gemäß der UCB-Heuristik ausgewählt, die Exploration und Exploitation balanciert. Die Expansion-Phase fügt neue Knoten hinzu, während die Simulation-Phase Zufallsspielzüge bis zum Endzustand durchführt. In der abschließenden Backpropagation-Phase werden die Ergebnisse dieser Simulationen entlang des durchlaufenen Pfades rückwärts propagiert und aktualisieren die Knotenwerte sowie Besuchszahlen.

Diese Methode wird iterativ implementiert und benötigt, im Gegensatz zum klassischen Minimax mit Tiefensuche, keine explizite Bewertungsfunktion. Während Minimax ohne Optimierung exponentielle Laufzeit von $O(b^d)$ aufweist, wobei $b = \text{mögliche Spalten pro Zug}$ und $d = \text{Zuege im Voraus}$, funktioniert MCTS praktikabler bei Spielen mit moderater Komplexität wie 4 Gewinn. Mit nur 32 MB Speicher und etwa 500.000 Simulationen erreicht MCTS zuverlässige Spielstärke. Damit bietet MCTS eine effiziente Alternative zu Alpha-Beta Pruning, insbesondere wenn keine domänenspezifische Evaluierungsfunktion vorhanden ist.

2.4 Wahl des Algorithmus

Für diese Implementierung ich mich für den **Minimax-Algorithmus mit Alpha-Beta-Pruning** entschieden [6, 3]. Diese Wahl wird durch folgende Faktoren begründet:

1. **Vorhersagbarkeit und Optimalität:** Minimax mit Alpha-Beta-Pruning garantiert die Findung des optimalen Zuges unter Annahme optimalen gegnerischen Spiels, sofern die Suchtiefe ausreichend ist [6].
2. **Deterministische Spielweise:** Das Verfahren liefert reproduzierbare Ergebnisse, was für Analysen und Tests vorteilhaft ist.
3. **Effiziente Komplexitätsreduktion:** Mit Alpha-Beta-Pruning kann die Komplexität quadratisch reduziert werden, was tiefere Suchbäume ermöglicht [3].
4. **Domänenspezifische Optimierung:** Für Vier Gewinn kann eine spezialisierte Bewertungsfunktion entwickelt werden, die strategische Ziele quantifiziert [6].

3 Entwurf

4 Realisierung des Entwurfs

% !TEX root = ../termpaper.tex % @author Leonhard Lüdeke %

5 Analyse

6 Fazit

Literatur

- [1] ALLEN, James D.: Expert play in connect-four. In: *Verfügbar unter <https://web.archive.org/web/20131023004851/http://homepages.cwi.nl/~tromp/c4.html>* Allis 1988 (1990)
- [2] DENGEL, Andreas ; BERNS, Karsten ; BREUEL, Thomas M. ; BOMARIUS, Frank ; ROTH-BERGHOFER, Thomas R.: *KI 2008: Advances in Artificial Intelligence: 31st Annual German Conference on AI, KI 2008, Kaiserslautern, Germany, September 23-26, 2008, Proceedings*. Bd. 5243. Springer Science & Business Media, 2008
- [3] KNUTH, Donald E. ; MOORE, Ronald W.: An analysis of alpha-beta pruning. In: *Artificial Intelligence* 6 (1975), Nr. 4, S. 293–326. – URL <https://www.sciencedirect.com/science/article/pii/0004370275900193>. – ISSN 0004-3702
- [4] KOCSIS, Levente ; SZEPESVÁRI, Csaba: Bandit based Monte-Carlo Planning. In: *Proceedings of the European Conference on Machine Learning (ECML)*, Springer, 2006, S. 282–293
- [5] OWEN, Guillermo: *Spieltheorie*. Springer-Verlag, 2013
- [6] RUSSELL, Stuart J. 1. ; NORVIG, Peter 1. (Hrsg.): *Artificial intelligence a modern approach*. 3. ed., international ed. Boston Munich u.a. : Pearson, 2010 (Prentice-Hall series in artificial intelligence). – URL <http://www.gbv.de/dms/tib-ub-hannover/627596169.pdf>. – Literaturverz. S. 1063 - 1093

- [7] WIKIPEDIA: *Vier gewinnt* — *Wikipedia, The Free Encyclopedia*. <http://de.wikipedia.org/w/index.php?title=Vier%20gewinnt&oldid=260829991>. 2025. – [Online; accessed 26-November-2025]

Erklärung zur selbständigen Bearbeitung

Hiermit versichere ich, dass ich die vorliegende Arbeit in allen Teilen selbstständig angefertigt und keine anderen als die in der Arbeit angegebenen Quellen und Hilfsmittel benutzt habe. Die in meiner Arbeit verwendeten KI-basierten Hilfsmittel habe ich (ggf. mit Produktnamen) angegeben.

Ich verantworte die Übernahme jeglicher von mir verwendeter maschinell generierter Passagen vollumfänglich selbst und trage die Verantwortung für eventuell durch die KI generierte fehlerhafte oder verzerrte Inhalte, fehlerhafte Referenzen, Verstöße gegen das Datenschutz- und Urheberrecht oder Plagiate.

Mir ist bewusst, dass wahrheitswidrige Angaben als Täuschungsversuch behandelt werden können.

Ort

Datum

Unterschrift im Original